

LAB3 INTRODUCTION TO MARKOV MODELS

Exercise 1: Draw the Markov Chain, including the transition probabilities. Try the code and observe the output. Does it make sense?

```
jina@kitty:~/decision_making$ /bin/python3 /home/jin/decision_making/lab3_markov/EX1.py
States visited: [0, 0, 0, 1, 1, 2, 2, 2, 0, 1, 2, 2, 2, 2, 1, 0]
jina@kitty:~/decision_making$
```

It does make sense, while in state A (0) it tends to stay in A, but once it passes to state B, it stays a couple of times until it switches to state C, in state C, it has more tendency to stay there but since we are working with probabilities it can always choose other state, like in state A.

Exercise 2: A robot is moving in a 2x2 grid. In each step, the robot can move 1 cell up, down, left, or right. Model the behavior of the robot using a Markov Chain. Consider the following conditions:

- The probability of remaining in the same cell is 10%.
- The probability of successfully moving one cell (up, down, left, or right) is 40%.
- The probability of ending in a non-desired cell is 10%.
- The robot cannot move outside the grid.
- Initially, the robot is in cell (1,1). Draw the corresponding Markov Chain, define the transition matrix, and simulate the chain for 10 steps. How many times did the robot end up in a non-desired cell?

```
jina@kitty:~/decision_making$ /bin/python3 /home/jin/decision_making/lab3_markov/EX2.py
States visited: [3, 1, 0, 2, 0, 1, 0, 1, 3, 2, 0]
jina@kitty:~/decision_making$
```

It starts in state 3 which corresponds to cell (1,1).

It never goes diagonally so it never ends up in an undesired state and also never stays the same place since the probability of moving, in combination is 80 percent.

Exercise 3: Complete the transition probability matrix and the reward matrix. Ensure that the transition probabilities for each action from each state add up to 1.

```
P[0] = np.array([ # Transition probabilities for action: UP
    [0.90, 0.03, 0.04, 0.03], # moving up from state (0,0)
    [0.05, 0.90, 0.05, 0.00], # moving up from state (0,1)
    [0.90, 0.05, 0.05, 0.00], # moving up from state (1,0)
    [0.02, 0.90, 0.04, 0.04] # moving up from state (1,1)
])
P[1] = np.array([ # Transition probabilities for action: Down
    [0.03, 0.03, 0.9, 0.04], # moving down from state (0,0)
    [0.03, 0.03, 0.04, 0.9], # moving down from state (0,1)
    [0.04, 0.03, 0.9, 0.03], # moving down from state (1,0)
    [0.03, 0.04, 0.03, 0.9] # moving down from state (1,1)
])
P[2] = np.array([ # Transition probabilities for action: Left
    [0.9, 0.03, 0.03, 0.04], # moving left from state (0,0)
    [0.9, 0.03, 0.04, 0.03], # moving left from state (0,1)
    [0.04, 0.03, 0.9, 0.03], # moving left from state (1,0)
    [0.03, 0.04, 0.9, 0.03] # moving left from state (1,1)
])
P[3] = np.array([ # Transition probabilities for action: right
    [0.03, 0.9, 0.03, 0.04], # moving right from state (0,0)
    [0.03, 0.9, 0.04, 0.03], # moving right from state (0,1)
    [0.04, 0.03, 0.03, 0.9], # moving right from state (1,0)
    [0.03, 0.04, 0.03, 0.9] # moving right from state (1,1)
])
```

The reward matrixes are the same for each of the actions.

Exercise 4: Observe the output from the previous code. Describe the optimal policy and the state values. Then, change the reward matrix to include a penalty of -10 for any action that moves the robot to state (1,0). What happened to the optimal policy and the state values? Why?

```
jin@kitty:~/decision_making$ /bin/python3 /home/jin/decision_making/lab3_markov/EX3.py
Using the PolicyIteration method
Optimal Policy: (3, 1, 3, 1)
State Values: (7.905617522067043, 8.765601919614733, 8.757931097436408, 8.765670957014338)
```

In state 0, wants to go right (action3), arriving at state 1, wants to go down reaching the goal, if it's in the state 2 it will go right to reach the goal, and while in the goal it will try to go down to stay in the same state, however it can also use action 3 to go right since both actions have the same probability of staying in the same place given the state 3.

Looking at the state value the worst one is state 0 since it needs 2 movements to arrive at the goal.

```
R[0] = np.array([ #
    [0, 0, -10, 1],
    [0, 0, -10, 1],
    [0, 0, -10, 1],
    [0, 0, -10, 1]
])
R[1] = np.array([
    [0, 0, -10, 1],
    [0, 0, -10, 1],
    [0, 0, -10, 1],
    [0, 0, -10, 1]
])
R[2] = np.array([
    [0, 0, -10, 1],
    [0, 0, -10, 1],
    [0, 0, -10, 1],
    [0, 0, -10, 1]
])
R[3] = np.array([
    [0, 0, -10, 1],
    [0, 0, -10, 1],
    [0, 0, -10, 1],
    [0, 0, -10, 1]
])
```

After changing the reward matrixes.

```
Using the PolicyIteration method
Optimal Policy: (3, 1, 3, 1)
State Values: (4.772574264871667, 5.610160654508487, 5.701797644095473, 5.709335921602204)
```

while the state values changed a little bit, it also tells us that state 0 is the worst one to be in, and the optimal policy remained untouched because in states other than 0, you can reach the goal via 1 movement.

Exercise 5: Use an MDP to define what the robot must do in each cell to reach the goal in a 3x3 grid world. Calculate the optimal policy and the state values. Comment on the results obtained. What's the path followed by the robot starting from cell (2,2) to reach the goal according to the optimal policy?

```
Using the PolicyIteration method
Optimal Policy: (3, 3, 0, 0, 0, 0, 0, 0, 0)
State Values: (5.479035785156255, 6.2540357851562565, 6.2540357851562565, 4.938473285156256, 5.479035785156255, 6.2540357851562565, 4.561430941406256, 4.938473285156255, 5.479035785156257)
```

According to the policy, all states except 0 and 1, will choose up, while state 0 and 1 will choose right. The worst state value refers to state 6 which is the farthest from goal, while state 8 and 0 have the same value given they both need 2 movements to reach the goal.

To sum up states that need 1 movement have value: 6.25

states that need 2 have values : 5.4

states that need 3 have values: 4.9

and states that need 4 have value 4.5 .

The path followed by (2,2) is just going up.

Exercise 6: Cells (1,1) and (1,2) correspond to difficult terrain, and when the robot enters this area, it receives a penalty of -1. Adapt the MDP to consider this situation and compare the new optimal policy and state values with the previous one. What's the new path from cell (2,2)? What happens if you increase the penalty to -10? Why?

For penalty -1

```
Using the PolicyIteration method
Optimal Policy: (3, 3, 0, 0, 0, 0, 0, 2, 0)
State Values: (4.660654174990238, 5.435654174990239, 5.435654174990239, 4.120091674990239, 4.660654174990238, 5.43565417499024, 3.7430493312402393, 3.480062296474614, 3.8856541749902402)
```

The path remains unchanged, it just chooses to go up.

For penalty -10

```
Using the PolicyIteration method
Optimal Policy: (3, 3, 0, 0, 0, 0, 0, 2, 2)
State Values: (0.028123474555689247, 0.8031234745556892, 0.8031234745556892, -0.5124390254443113, 0.02812347455568903, 0.8031234745556891, -0.8894813691943114, -1.152468403959367, -1.33590186070896)
```

Penalty is great enough to be considered and now avoid those states completely, the path from 2,2 goes right and then up, then left circling around the penalty states.

Exercise 7: Adapt your MDP to avoid hitting the walls by changing the reward matrix.

```
Using the PolicyIteration method
Optimal Policy: (3, 3, 2, 0, 0, 0, 0, 2, 2)
State Values: (-2.14064891042375, -1.684095154900922, -2.14064891042375, -2.4590951549009223, -2.1406489104237494, -1.684095154900922, -2.68121141042375, -2.8361374986509222, -2.944198445189375)
```

This changes how the goal state acts. In previous experiences when arriving at this state, the actions chosen were up or right to hit the wall and have an 80 percent of staying in the same spot, now it chooses to go left and land in state 1 which always goes right to end up in goal again.

